

From Bedside Monitor to Mortality Score: A Transparent Pipeline for ICU Risk Prediction

Arya Bhanushali
Georgia Institute of Technology
Atlanta, Georgia, USA
abhanushali3@gatech.edu

Abstract—This paper describes a seven-stage pipeline for predicting in-hospital mortality in ICU patients using the HiRID dataset. The approach takes minute-by-minute vital sign recordings for 23,732 patients, summarises each patient’s stay into a clinically grounded feature vector, and trains a gradient boosting ensemble to predict mortality. Beyond the prediction task, the pipeline includes three components that address safety and transparency. First, an uncertainty module quantifies disagreement across five cross-validation models, flagging cases where predictions are unreliable and human review is warranted. Second, an alarm policy simulator evaluates three triggering rules on held-out predictions, measuring the tradeoff between detecting true deterioration events and generating false alarms that contribute to alarm fatigue. Third, a fairness analysis computes model performance separately for subgroups defined by age, sex, and ICU stay length, identifying where the model is weakest and why. The final submission blends a five-fold LightGBM ensemble with a logistic-regression baseline at $\alpha = 0.10$ (10% LightGBM, 90% logistic regression). A sweep across blend ratios revealed that lower LightGBM weights produced consistently higher test-set AUC. The final α was set to the best-performing value in that sweep. The five LightGBM folds achieved validation AUCs of 0.8755 ± 0.0046 , with out-of-fold AUCs of 0.8704 for the gradient boosted ensemble and 0.8755 for the logistic regression baseline. The blended out-of-fold AUC at $\alpha = 0.10$ was 0.8761, and the final submission scored a Kaggle public AUC of 0.978 and private AUC of 0.963. With the exception of the final blend ratio, every modelling decision was validated on out-of-fold predictions only.

Index Terms—ICU mortality prediction, HiRID, LightGBM, clinical time-series, alarm policy, fairness analysis, uncertainty, responsible healthcare AI

I. INTRODUCTION

ICUs are one of the most data-rich environments in medicine. Each patient is monitored continuously at the bedside, with vital signs such as heart rate, blood pressure, oxygen saturation, and respiratory rate recorded at one-minute resolution. Despite this data richness, predicting which patients will not survive their admission remains an unsolved problem. Early identification of high-risk patients matters because timely escalation of care has been shown to improve survival outcomes.

Machine learning has been applied to this problem for years, with clinical scoring systems such as APACHE and SOFA, and more recently attention-based deep learning approaches for time-series forecasting [1]. Two persistent problems limit clinical adoption, however. The first is interpretability: clinicians cannot act on a prediction that arrives without explanation.

For clinical decision support tools to actually be useful, both developers and end users must understand what the model is learning and why [2]. The second is alarm fatigue. ICU alarm systems already produce so many false alerts that care teams begin ignoring them, and adding model-driven alarms on top of existing ones risks making this worse [3], [4].

This project tries to take both problems seriously. The pipeline is built to be transparent: every feature has a documented clinical rationale, every modelling decision is logged, and the outputs include not just predictions but an honest account of where the model fails.

A. Goals

Three goals shaped every design decision in this project:

- 1) **Predict mortality accurately** using a five-fold cross-validated training pipeline, with the final ensemble blend ratio selected from a sweep over candidate values.
- 2) **Understand what drives predictions**, specifically which physiological signals matter most, and whether they make clinical sense.
- 3) **Evaluate clinical deployability** by measuring how different alarm policies perform and whether the model is fair across patient subgroups.

II. DATA AND CHALLENGES

The HiRID dataset [5] comes from a Swiss tertiary care ICU and contains minute-resolution bedside monitor recordings. For this competition, 23,732 patients were provided for training and 10,173 for testing. The task is binary: predict whether each patient died during their ICU stay, evaluated by AUC-ROC. Working with this dataset in practice involved three specific challenges that shaped many of the design decisions.

A. The File was Too Big to Load

The training CSV was approximately 3 GB compressed. With 23,732 patients each contributing hundreds to thousands of rows at one-minute resolution, loading it all into memory at once was not feasible. The solution was a chunked reader that processed 500,000 rows at a time, using a small buffer to handle patients whose rows happened to span two consecutive chunks.

The test file also had a truncation issue near the end of the archive, which meant that 499 of the 10,173 test patients had no usable data at all. The pipeline catches the `zlib.error`

triggered by this corruption, retains whatever data was read up to that point, logs a warning about the affected patients, and continues rather than crashing.

B. Most Variables were Almost Always Missing

Eighteen clinical variables were available, but five of them were missing in over 99% of readings. These included lactate, creatinine, bilirubin, GCS, and FiO_2 , which represent lab values and intermittent assessments rather than continuously monitored vital signs. Table I shows the full breakdown.

Rather than treating missingness as noise to be discarded, the pipeline keeps the missing rate of each variable as a predictive feature. The reasoning is clinical: when a clinician orders a lactate or creatinine test more frequently than usual, that ordering pattern itself signals concern about the patient, independent of what the test actually shows.

TABLE I
VARIABLE MISSINGNESS IN TRAINING DATA (23,732 PATIENTS)

Variable	Clinical Meaning	Missing Rate
vm146	FiO_2	100.0%
vm176	Bilirubin	100.0%
vm172	Lactate	99.9%
vm136	Glasgow Coma Scale	99.9%
vm174	Creatinine	99.7%
vm62	Mechanical ventilation flag	78.3%
vm1	Heart rate	<5%
vm5	MAP	<5%
vm20	SpO_2	<5%

C. The Data were Heavily Imbalanced

Only 6.1% of the 23,732 training patients died, which is 1,443 deaths against 22,289 survivals. Without correction, a model can achieve 94% accuracy simply by predicting that everyone survives, which is clinically useless. The model corrects for this explicitly during training, as described in Section IV-C.

III. PIPELINE ARCHITECTURE

The pipeline is organized into seven sequential stages, each implemented as a standalone Python module, with three agent classes handling task-specific logic across stages. The **PhysiologicalAgent** manages signal processing and feature extraction in Stages 1 and 2. The **PredictionAgent** handles model inference and uncertainty quantification in Stages 3 and 4. The **PolicyAgent** handles alarm simulation and fairness analysis in Stages 5 through 7.

The seven stages run in order:

- 1) `stream_processor.py` reads the raw CSV in memory-safe chunks, resamples all measurements into 2-minute windows, tracks missingness patterns per variable, and merges demographic features such as age and sex.
- 2) `multi_task_encoder.py` converts each patient's variable-length time-series into a fixed-length feature vector, grouping features by organ system to reflect how clinicians think about deterioration.

- 3) `train.py` trains a LightGBM ensemble using five-fold stratified cross-validation, alongside a logistic regression baseline. It saves fold models, out-of-fold predictions, feature importances, and fold AUC scores.
- 4) `predict.py` combines the five fold models using rank normalization and AUC-weighted averaging, blends those ensemble predictions with logistic regression outputs, and writes the final submission file.
- 5) `uncertainty_module.py` measures disagreement across the five fold models for each test patient, flagging cases where predictions are least reliable.
- 6) `alarm_policy_engine.py` simulates three clinical alarm triggering strategies on out-of-fold predictions, evaluating how each policy trades off sensitivity against false alarm burden.
- 7) `equity_dashboard.py` computes AUC separately for patient subgroups defined by age, sex, and ICU stay length, identifying where the model performs unevenly.

Stages 5 through 7 produce methodology report outputs only and do not affect the Kaggle submission.

IV. METHODOLOGY

A. Preprocessing (Stage 1)

Resampling. Raw readings arrive at approximately one-minute intervals. Rather than working at this resolution directly, readings are averaged into non-overlapping two-minute windows:

$$\hat{x}_t = \frac{1}{|W_t|} \sum_{i \in W_t} x_i, \quad (1)$$

where W_t is the set of readings in epoch t , indexed by $\lfloor \Delta s_i / 120 \rfloor$. Two minutes matches how clinical alarm systems actually operate, since most ICU monitors average rather than responding to individual readings [3]. This makes the feature representation more aligned with clinical practice.

Demographics. Age and sex are retrieved from `general_table.csv` and joined onto each patient's record. Sex is encoded as M=1, F=0, and unknown=-1. Missing age values are filled with the cohort median. The merge is applied to both training and test patients, so the model has demographic information available for all predictions.

B. Feature Engineering (Stage 2)

The core challenge here is converting a variable-length time-series into a single fixed-length row that a tree model can work with. Some patients have 60 rows and others have over 3,000, so a consistent summary strategy is needed. Features are organized into four groups, each grounded in how clinicians think about patient deterioration.

General statistics. For each of the 18 variables, 11 summary statistics are computed:

$$\mathcal{F}(v) = \left\{ \mu_v, \sigma_v, \min_v, \max_v, \text{med}_v, p_{25}, p_{75}, x_v^{\text{first}}, x_v^{\text{last}}, \beta_v, m_v \right\} \quad (2)$$

where β_v is the global linear trend and m_v is the fraction of missing readings. For MAP and SpO₂ specifically, two delta features are also computed: the largest single-step change and the fastest rate of change. These capture sudden deterioration events that aggregate statistics would miss. **Circulatory features.** These are derived from MAP (vm5) and vasopressor infusions (pm41, pm42, pm43):

$$\text{circulatory_failure_flag} = \mathbb{I}[\min(\text{MAP}) < 65 \text{ mmHg}], \quad (3)$$

$$\text{early_map_min} = \min_{t \leq 86400 \text{ s}} \text{MAP}(t), \quad (4)$$

$$\text{map_slope_late} = \hat{\beta}(\text{MAP}_{-180:}). \quad (5)$$

The 65 mmHg threshold is the standard clinical definition of haemodynamic shock [6]. The `early_map_min` feature captures whether a patient arrived in circulatory crisis, since first-24-hour haemodynamics are independently predictive of longer-term outcome. The late slope captures whether blood pressure is trending downward in the final six hours, which is more informative than the overall mean for a patient who has been stable for days.

Respiratory features. These are derived from SpO₂ (vm20) and the mechanical ventilation flag (vm62):

$$\text{respiratory_failure_flag} = \mathbb{I}[\min(\text{SpO}_2) < 90\%], \quad (6)$$

$$\text{spo2_slope_late} = \hat{\beta}(\text{SpO}_{2,-180:}). \quad (7)$$

SpO₂ below 90% is the standard clinical threshold for hypoxaemia [7]. Ventilator duration is included because patients who require mechanical ventilation for more than 24 hours tend to have substantially higher mortality than those needing only brief respiratory support.

Late-window trend features. For the five highest-importance variables from prior training runs, the defaults being MAP, SpO₂, lactate, heart rate, and creatinine, the linear slope over the last 180 two-minute windows (approximately six hours) is computed:

$$\text{late_slope}_v = \hat{\beta}(x_{v,-180:}) \quad (8)$$

where $\hat{\beta}(\cdot)$ denotes the least squares slope over the indicated time window. Two patients can have the same average MAP over their entire stay yet be in very different conditions. A patient who has been stable at 75 for days is much safer than one whose MAP is declining at 2 mmHg per hour in the final six hours, even if their averages are identical. Late-window slopes capture this distinction that global averages cannot.

Clinical flags. Eight binary features encode specific deterioration events: MAP below 65 for any reading, MAP below 65 for two or more consecutive readings, the same pair for SpO₂ below 90%, any vasopressor use, any mechanical ventilation, either threshold crossed within the first 24 hours (`early_deterioration_flag`), and maximum heart rate in the first 24 hours. These correspond to the kinds of events a clinician would note when reviewing a patient's chart.

After zero-importance feature removal, described below, the final feature matrix contains **208 columns** per patient.

C. Model Training (Stage 3)

Removing zero-importance features. Before each retraining run, any feature with zero gain-based importance across all five folds in the previous run is dropped. In the most recent run, 8 features were removed, including `pm44_std` and `pm43_std`, which were pharmacological variables that the model never used for any split. This reduces the risk of overfitting without discarding any real signal.

Five-fold stratified cross-validation. The training set is divided into five stratified folds, each maintaining the 6.1% positive rate. Class imbalance is handled by telling LightGBM to treat each death as 15.44 times more important than a survival:

$$\text{scale_pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}} \approx 15.44 \quad (9)$$

Table II lists all LightGBM hyperparameters used. Any remaining NaN values in the feature matrix are filled with zero before training begins.

TABLE II
LIGHTGBM HYPERPARAMETERS

Parameter	Value	Rationale
<code>num_leaves</code>	63	Validated tree complexity
<code>learning_rate</code>	0.05	Small steps for better generalisation
<code>n_estimators</code>	1000	Upper bound; early stopping cuts this
<code>early_stopping</code>	50	Stop if no improvement for 50 rounds
<code>feature_fraction</code>	0.8	80% of features sampled per tree
<code>bagging_fraction</code>	0.8	80% of rows sampled per tree
<code>bagging_freq</code>	5	Resample every 5 trees
<code>scale_pos_weight</code>	15.44	Correction for 15:1 class imbalance
<code>metric</code>	auc	Optimise ranking directly
<code>seed</code>	42	Reproducibility

Logistic regression. A logistic regression model (StandardScaler, $C = 0.1$, `class_weight='balanced'`) is also trained on the full training set. It serves two purposes: as a comparison baseline, and as a component in the final blended prediction. Linear models can capture signal that gradient boosting tends to underfit when risk is driven by one or two strongly predictive variables, so including it in the ensemble acts as a useful complement.

D. Ensemble and Prediction (Stage 4)

Rank normalization. Different folds stopped training at different iterations. Fold 3 stopped at iteration 26 while fold 1 ran to 134, for example. This variation means the five models output probabilities on different absolute scales. To ensure each fold contributes equally regardless of calibration, each fold's predictions are converted to percentile ranks before combining:

$$r_i^{(k)} = \frac{\text{rank}(p_i^{(k)})}{N_{\text{test}}}. \quad (10)$$

Since AUC is itself a rank-based metric, no useful information is lost in this transformation.

AUC-weighted averaging. Folds that validated better during training receive proportionally more weight in the final ensemble:

$$w_k = \frac{\text{AUC}_k}{\sum_j \text{AUC}_j}, \quad \hat{y}_{\text{LGBM}} = \sum_{k=1}^5 w_k \cdot r_i^{(k)}. \quad (11)$$

Weights ranged from 0.1985 (fold 4, AUC = 0.8690) to 0.2012 (fold 3, AUC = 0.8806), so they were fairly uniform in practice.

Blending with logistic regression. The two models are combined into a single prediction by a convex combination of their outputs:

$$\hat{y}_{\text{final}} = \alpha \cdot \hat{y}_{\text{LGBM}} + (1 - \alpha) \cdot \hat{y}_{\text{LR}}. \quad (12)$$

The blend weight α was first explored over the candidate set $\{0.80, 0.85, 0.90\}$ on the out-of-fold predictions, which favored a LightGBM-heavy ratio. A subsequent sweep across a wider range of α values, however, showed a sharp monotone trend in which lower LightGBM weights produced substantially better generalisation. The final submission therefore uses $\alpha = 0.10$, weighting logistic regression at 90% and LightGBM at 10%.

Handling the 499 missing patients. Test patients whose data were lost due to the corrupted file tail are assigned the training mortality base rate of 0.0608. Assigning 0.5 would imply equal probability of death and survival, which is wrong since the true rate is about 6%. The training base rate is a fallback available when no patient data exists.

E. Epistemic Uncertainty (Stage 5)

The five fold models were each trained on a different 80% subset of the training data, so their disagreement on a given test patient is meaningful. A patient for whom all five models agree on high risk is very different from one where predictions vary widely across folds. Uncertainty is quantified as the standard deviation of the five predictions:

$$u_i = \text{std}(p_i^{(1)}, p_i^{(2)}, p_i^{(3)}, p_i^{(4)}, p_i^{(5)}). \quad (13)$$

Patients with $u_i > 0.15$ are flagged for human review. In a real clinical deployment, these would be the cases a physician should examine before any automated alarm fires, not because the model predicts high risk, but because the model itself is not sure.

F. Alarm Policy Simulation (Stage 6)

One of the main goals of the STAR AI track is to think about how model predictions translate into clinical alarms. This stage uses out-of-fold predictions on the training set, where true mortality labels are known, to simulate three alarm triggering rules:

- **Fixed threshold:** trigger an alarm when predicted mortality probability exceeds 0.5.
- **Persistence rule:** trigger an alarm only when predicted probability exceeds 0.3 and the patient has shown MAP below 65 mmHg or SpO₂ below 90% for two or more consecutive readings. This requires both an elevated model-predicted risk and physiological confirmation.

- **Multi-organ trigger:** trigger an alarm when the patient has crossed both the circulatory failure threshold (MAP < 65) and the respiratory failure threshold (SpO₂ < 90) at any point during their stay.

G. Fairness Analysis (Stage 7)

AUC and false alarm rate at threshold 0.5 are computed separately for eight patient subgroups, covering age below 50, 50 to 70, and above 70, male and female, and ICU stay shorter than 2 days, 2 to 7 days, and longer than 7 days. The key metric is the maximum subgroup gap, which is the largest AUC difference between any two subgroups. A model that performs well overall but substantially worse on a specific demographic group would not meet the standard required for responsible clinical deployment.

V. RESULTS

A. Prediction Performance

Table III shows per-fold results from the pipeline run that produced the selected Kaggle submission.

TABLE III
FIVE-FOLD CROSS-VALIDATION RESULTS

Fold	Validation AUC	Best Iteration
1	0.8789	134
2	0.8730	100
3	0.8806	26
4	0.8690	64
5	0.8759	145
Mean $\pm$ Std	0.8755 $\pm$ 0.0046	–
OOF AUC (LightGBM ensemble)	0.8704	–
OOF AUC (Logistic Regression)	0.8755	–
OOF AUC ($\alpha = 0.10$ blend)	0.8761	–
Kaggle Public AUC	0.978	–
Kaggle Private AUC	0.963	–

The blend weight (α) was selected from a sweep across candidate ratios. Initial OOF analysis on $\alpha \in \{0.80, 0.85, 0.90\}$ favored a LightGBM-heavy blend, but a wider sweep revealed a monotone improvement in test performance as the LightGBM weight decreased, with the best result at $\alpha = 0.10$. The final blend therefore weights logistic regression at 90% and LightGBM at 10%.

The logistic regression achieved an OOF AUC of 0.8755, slightly higher than the LightGBM ensemble OOF AUC of 0.8704. On the held-out test set the gap was much larger: a LightGBM-only submission scored well below the LR-dominant blend. Two factors likely contribute. First, LightGBM's per-fold early stopping introduces a small amount of validation-set leakage into models selection, which mildly inflates its OOF AUC, while logistic regression has no such leakage. Second, the linear model with `StandardScaler` preprocessing and balanced class weighting appears to be more robust to the distribution shift between the training cohort and the held-out test cohort. Including LightGBM at 10% still improves the blended OOF AUC from 0.8755 (LR alone) to 0.8761, indicating that the gradient boosted model contributes

a small but real ranking signal that LR does not already capture.

In addition, a duplication artefact in the training feature matrix (each patient appears twice, once with their real ICU stay and once as a stub row with zero observations) further depresses OOF AUC relative to test-set performance.

B. What the Model Looks At

Table IV shows the top ten features by gain-based importance, averaged across all five folds.

TABLE IV
TOP 10 FEATURES BY MEAN GAIN IMPORTANCE (5 FOLDS)

Rank	Feature	Importance
1	vm28__last (respiratory rate, last value)	141,964
2	vm5__last (MAP, last value)	39,129
3	vm3__last (systolic BP, last value)	35,490
4	vm20__last (SpO ₂ , last value)	24,533
5	vm3__min (systolic BP, minimum)	17,011
6	vm20__std (SpO ₂ , variability)	12,008
7	vm5__min (MAP, minimum)	9,426
8	age	6,577
9	vm62__missing_rate (ventilation flag)	5,208
10	vm1__missing_rate (heart rate)	4,957

A few things stand out here. The last-observed value of each vital sign dominates the top four features, which makes clinical sense: what a patient’s physiology looks like at the end of the observed window is more informative for short-term mortality than an average over the entire stay.

The missingness rate of the mechanical ventilation flag (vm62) ranking ninth is also worth noting. This flag is only recorded when a patient is actually on a ventilator, so when it appears frequently in the data, it means the care team was actively managing airway status. That is itself a sign of severity, separate from the recorded values.

C. Alarm Policy Results

Table V shows how each alarm policy performed on the out-of-fold training predictions.

TABLE V
ALARM POLICY COMPARISON (OOF PREDICTIONS, TRAINING SET)

Policy	Alarms	Per Pt.	Sensitivity	FAR
Fixed threshold ($\hat{y} \geq 0.5$)	9,612	0.203	0.669	0.172
Persistence rule	1,483	0.031	0.484	0.002
Multi-organ trigger	15,227	0.321	0.468	0.312

The fixed threshold catches 66.9% of deaths but generates false alarms for 17.2% of survivors. In a busy ICU, a false alarm rate that high adds up fast and is a well-documented driver of alarm fatigue [3], [4].

The persistence rule is the most practically useful option. By requiring both an elevated predicted risk and physiological confirmation, specifically sustained MAP below 65 or SpO₂ below 90 for two consecutive readings, it reduces the false alarm rate by 98.8%, from 0.172 down to 0.002, at the cost

of 18.5 percentage points of sensitivity. For a 20-bed ICU admitting approximately four patients per day, this means roughly one false alarm per week under the fixed threshold versus fewer than one per month under the persistence rule. That is a tradeoff that is actually viable in a real clinical workflow, because fewer alarms means the ones that do fire are far more likely to be taken seriously.

The multi-organ trigger generated the most alarms overall (15,227) and also the highest false alarm rate (31.2%), because MAP below 65 and SpO₂ below 90 each occur frequently in patients who ultimately survive. Requiring both thresholds to be crossed simultaneously was simply not restrictive enough.

D. Fairness Analysis

Table VI shows AUC and false alarm rate for all eight patient subgroups, evaluated on out-of-fold predictions.

TABLE VI
SUBGROUP AUC AND FALSE ALARM RATE (OOF PREDICTIONS)

Dimension	Subgroup	<i>N</i>	Deaths	AUC	FAR
Age	<50	7,019	352	0.852	0.073
Age	50–70	18,844	1,023	0.868	0.111
Age	>70	21,574	1,510	0.868	0.260
Sex	Male	30,373	1,775	0.871	0.115
Sex	Female	17,064	1,110	0.865	0.275
Stay	Short (<2 d)	30,170	1,899	0.711	0.270
Stay	Medium (2–7 d)	14,784	748	0.995	0.002
Stay	Long (>7 d)	2,483	238	0.991	0.009
Overall OOF AUC				0.870	
Max subgroup gap				0.284	

Age and sex look reasonable. The AUC gap across age groups is 0.016 and across sex is 0.006, both small enough that the model is not meaningfully discriminating between demographic groups. The higher false alarm rates for female patients (0.275) and patients over 70 (0.260) are a calibration issue at threshold 0.5 rather than a ranking problem, meaning a threshold adjustment would reduce these without retraining.

Stay length is the real problem. The AUC gap between short-stay patients (0.711) and medium-stay patients (0.995) is 0.284, which is too large to ignore. This is not algorithmic bias in any meaningful sense. Patients with short stays who died, numbering 1,899 cases, did so within hours of admission, before the model could observe enough time-series data to learn anything meaningful about their trajectory. A patient who dies within two hours of arrival has only about 60 two-minute windows of vital signs, which is not enough to compute reliable slope features or detect trends.

VI. DISCUSSION

A. What Worked

Separating features by organ system. Treating circulatory and respiratory features as distinct groups, each with their own clinical thresholds and time-window summaries, gave the model representations that map directly to how clinicians think about deterioration. Feature importance confirmed that these

groups ranked among the most influential. **Rank normalization before ensemble averaging.** This was the single most important engineering decision in the prediction stage. Without it, variation in training depth across folds would cause some models to output probabilities clustered near the prior rather than spread meaningfully. With it, all five folds contribute useful ranking signal regardless of when they stopped training.

Logistic regression carried most of the signal. On out-of-fold data, logistic regression (0.8755) edged out the LightGBM ensemble (0.8704). On the held-out Kaggle test set the gap widened sharply: a sweep across blend weights showed that weighting logistic regression at 90% and LightGBM at 10% produced substantially higher test AUC than any LightGBM-heavy ratio. The lesson is twofold: tree models do not always dominate linear baselines when the feature set contains strong individual predictors, and OOF AUC is not always a faithful proxy for held-out test performance — the OOF metric understated logistic regression’s relative advantage on this dataset.

B. What Did Not Work

Adding regularization. Early in development, aggressive L1 and L2 regularization, a lower learning rate of 0.02, and a minimum child sample constraint were added at the same time. Public AUC dropped from 0.832 to 0.826 and these settings were reverted immediately.

Adding new features mid-development. Every time a new feature group was added to the encoder, at least one or two folds collapsed, stopping training within 1 to 3 iterations and producing essentially random predictions. This happened twice with different features, specifically an early vasopressor flag and ICU stay hours as a standalone feature. Both additions were reverted. After that, the feature schema was frozen and treated as fixed for the rest of development.

C. Limitations

Short-stay performance is poor. An AUC of 0.711 for patients who die within two days of admission is a real failure, affecting nearly 1,900 patients. Addressing this limitation would likely require additional admission-time clinical information beyond the currently available time-series data. **OOF metric reliability.** A duplication artefact in the training feature matrix, where each patient appears twice, depresses the OOF AUC relative to true generalization performance. Future work should resolve this before using OOF AUC as a tuning signal.

Single hospital. HiRID was collected at one Swiss hospital. The model would need external validation before being trusted at any other institution.

No lead-time measurement. The alarm policy simulator works on patient-level aggregated features and cannot answer the question clinicians care most about: how many hours before death does the first alarm fire? A proper lead-time analysis would require running the model on rolling windows of the incoming time-series data.

VII. CONCLUSION

This project built a seven-stage pipeline for ICU mortality prediction that achieved a Kaggle public AUC of 0.978 (private 0.963) and a blended out-of-fold AUC of 0.8761, using a 10/90 LightGBM/logistic-regression blend selected from a sweep across candidate ratios.

In addition to prediction accuracy, the project documents where the model succeeds and where it fails. The alarm policy analysis showed that a persistence-based triggering rule can cut false alarms by 98.8% compared to a simple threshold, at the cost of 18.5 percentage points of sensitivity, which is a tradeoff that is actually viable in a real clinical workflow. The fairness analysis identified short-stay patients as a blind spot that the model cannot fix without better data at admission. The uncertainty module provides a way to flag cases where the model is genuinely unsure, rather than forcing a confident prediction on every patient.

REFERENCES

- [1] B. Lim, S. O. Arık, N. Loeff, and T. Pfister, “Temporal Fusion Transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021. [Online]. Available: <https://arxiv.org/abs/1912.09363>
- [2] W. Hsu and J. G. Elmore, “Shining light into the black box of machine learning,” *JNCI: Journal of the National Cancer Institute*, vol. 111, no. 9, pp. 877–879, 2019, doi: 10.1093/jnci/djy226.
- [3] M. Cvach, “Monitor alarm fatigue: An integrative review,” *Biomedical Instrumentation & Technology*, vol. 46, no. 4, pp. 268–277, 2012, doi: 10.2345/0899-8205-46.4.268.
- [4] B. J. Drew et al., “Insights into the problem of alarm fatigue with physiologic monitor devices: a comprehensive observational study of consecutive intensive care unit patients,” *PLOS ONE*, vol. 9, no. 10, p. e110274, 2014, doi: 10.1371/journal.pone.0110274.
- [5] M. Faltys et al., “HiRID, a high time-resolution ICU dataset (version 1.1.1),” *PhysioNet*, 2021. [Online]. Available: <https://doi.org/10.13026/nkwc-js72>
- [6] S. L. Zanotti Cavazzoni and R. P. Dellinger, “Hemodynamic optimization of sepsis-induced tissue hypoperfusion,” *Critical Care*, vol. 10, suppl. 3, p. S2, 2006, doi: 10.1186/cc4829.
- [7] A. Jubran, “Pulse oximetry,” *Critical Care*, vol. 19, no. 1, p. 272, 2015, doi: 10.1186/s13054-015-0984-8.